Hands-On Pertemuan 1 - Fondasi Web Security

Mata kuliah MK15 Keamanan Aplikasi Web - Institut Teknologi Tangerang Selatan, 2026/2027 Ganjil

Nama	Farel Eka Saputra	NIM	1002250027
Prodi	Teknologi Informasi	Tanggal praktikum	14 Oktober 2026

Nama	Fauzan atillah	NIM	1002250039
Prodi	Teknologi Informasi	Tanggal praktikum	14 Oktober 2026

Nama	Hafiz Hablillah	NIM	1003250012
Prodi	Teknologi Informasi	Tanggal praktikum	14 Oktober 2026

Lingkungan: Windows (PowerShell / Windows Terminal), Google Chrome dengan DevTools, Python 3.14.8 (python -m http.server), curl. Port yang dipakai: 8000 (Origin A) dan 127.0.0.1:8000 (Origin C). Seluruh percobaan hanya pada localhost / perangkat sendiri; tidak ada target publik atau data sensitif.

Tujuan

Praktikum ini bertujuan memahami fondasi web security: (1) browser dan curl sama-sama menjadi HTTP client yang mengirim request ke server lokal, (2) origin (scheme + host + port) menentukan isolasi localStorage, dan (3) elemen UI/DOM di browser dapat diubah pengguna sehingga bukan security boundary.

Ringkasan Temuan

Halaman lab dilayani lewat python -m http.server 8000 dan diamati dengan DevTools serta curl. Network panel mencatat GET http://localhost:8000, sedangkan curl.exe -i mendapat halaman yang sama dengan HTTP/1.0 200 OK dan Content-type: text/html tanpa memakai UI browser; curl juga bebas menambah header buatan sendiri (X-Student-Lab: week1) dan server tetap membalas 200.

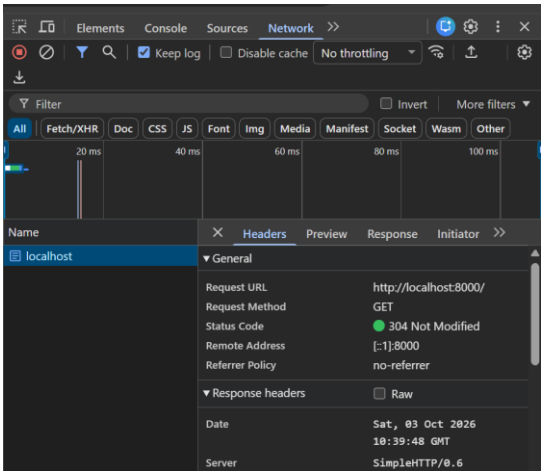
localStorage username=taufik dan role=student tersimpan di Origin A, tetapi pada http://127.0.0.1:8000 (Origin C) getItem('username') bernilai null karena host berbeda berarti origin berbeda. Input role yang awalnya disabled berhasil diaktifkan dan diubah menjadi admin lewat Console, sehingga terbukti bahwa DOM dan state client dapat dimodifikasi pengguna dan tidak boleh dipercaya server.

1. Langkah Kerja

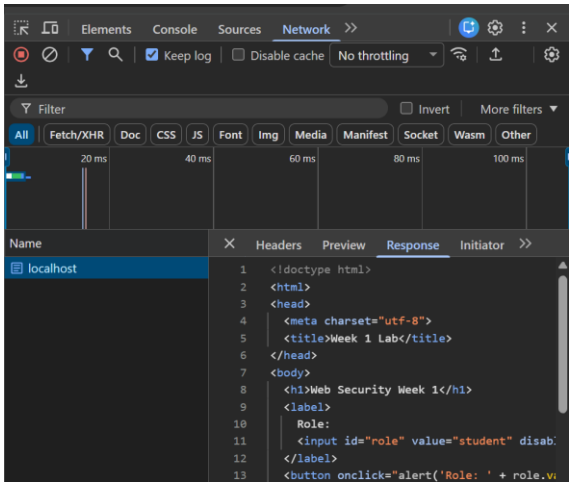
1. Buat folder dan file: mkdir websec-week1, cd websec-week1, lalu buat index.html sesuai modul (input role bernilai student dan disabled, tombol Submit).
2. Jalankan server: python -m http.server 8000, buka http://localhost:8000; Console menampilkan Origin: http://localhost:8000.
3. DevTools > Network > reload, lalu catat Request URL, Method, Status Code, dan header (Evidence 1).
4. Terminal kedua: curl.exe -i http://localhost:8000/ dan curl.exe -i -H "X-Student-Lab: week1" http://localhost:8000/ (Evidence 2).
5. Console Origin A: localStorage.setItem('username','taufik'), setItem('role','student'), getItem(...), location.origin; cek di Application > Local Storage. Lalu buka http://127.0.0.1:8000 dan jalankan localStorage.getItem('username') (Evidence 3).
6. Console: const role = document.querySelector('#role'), role.disabled = false, role.value = 'admin' (Evidence 4).

2. Evidence 1 - Network Panel (DevTools)

Request URL http://localhost:8000/, Method GET, Status 304 Not Modified (reload memakai cache karena file tidak berubah), Remote Address [::1]:8000, Referrer Policy no-referrer. Response header berisi Server: SimpleHTTP/0.6 Python/3.14.8, dan isi HTML terlihat di tab Response.



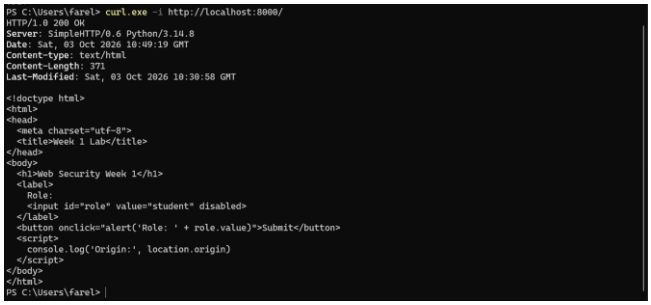
Gambar 1. Network > Headers: URL, method GET, status 304, header request/response.



Gambar 2. Network > Response: body HTML dari index.html (Week 1 Lab).

3. Evidence 2 - Perbandingan Browser dengan curl

curl.exe -i mengambil resource yang sama dengan status line HTTP/1.0 200 OK, Content-type: text/html, Content-Length: 371, lalu body HTML. Pada percobaan kedua, header X-Student-Lab: week1 dibuat langsung oleh client dan server tetap membalas 200 OK. Artinya request dapat dibuat di luar UI resmi, sehingga server tidak boleh menganggap semua request berasal dari form atau tombol yang disediakan.



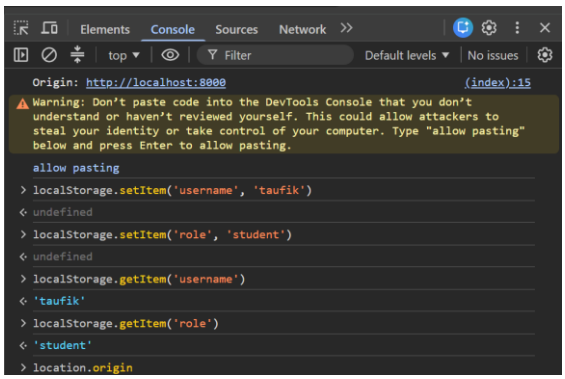
Gambar 3. curl.exe -i http://localhost:8000/ : 200 OK, Content-type text/html.



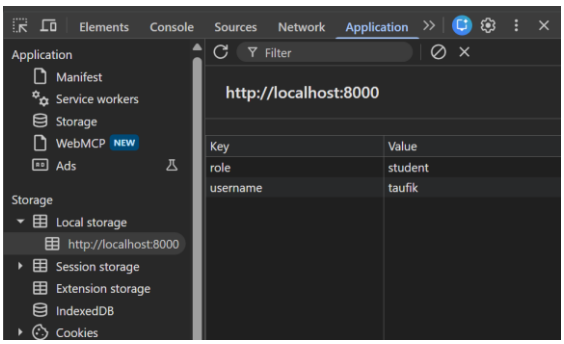
Gambar 4. curl.exe -i -H "X-Student-Lab: week1" ...: header buatan client, tetap 200 OK.

4. Evidence 3 - Isolasi localStorage Antar-Origin

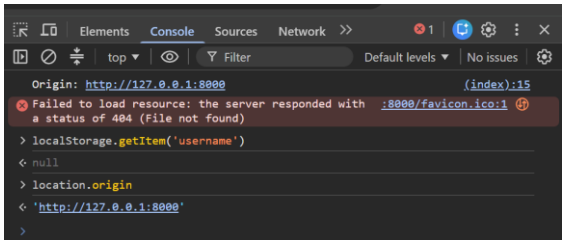
Pada Origin A data username=taufik dan role=student tersimpan dan terlihat di Application > Local Storage (Gambar 5-6). Pada Origin C (http://127.0.0.1:8000) nilai tersebut tidak tersedia (Gambar 7).



Gambar 5. Origin A: setItem/getItem username dan role, location.origin = http://localhost:8000.



Gambar 6. Application > Local Storage http://localhost:8000: role=student, username=taufik.



Gambar 7. Origin C (127.0.0.1:8000): getItem('username') = null.

URL	Scheme / host / port	Hasil username	Penjelasan
http://localhost:8000	http / localhost / 8000	taufik	Origin A, tempat data disimpan.
http://localhost:8001	http / localhost / 8001	null (penjelasan saja; tidak diuji)	Port berbeda, origin berbeda.
http://127.0.0.1:8000	http / 127.0.0.1 / 8000	null (Gambar 7)	Host berbeda, origin berbeda.

5. Evidence 4 - DOM Manipulation



Gambar 8. Console: role.disabled = false dan role.value = 'admin'; input Role kini berisi admin.

Observasi	Kesimpulan keamanan
Input disabled bisa diaktifkan ulang (role.disabled = false).	disabled hanya kontrol UX, bukan authorization.
Nilai role berubah dari student menjadi admin di DOM.	Data dari browser harus dianggap attacker-controlled.
Perubahan hanya terjadi di browser sendiri.	Server yang harus memutuskan hak akses.

6. Analisis (Jawaban Q1-Q6)

Pertanyaan	Jawaban
Q1. Mengapa Origin B tidak bisa melihat localStorage Origin A?	Origin = scheme + host + port. Port 8001 berbeda dari 8000, sehingga origin berbeda. Browser menyimpan localStorage per origin (Same-Origin Policy), jadi data Origin A tidak terbaca dari Origin B.
Q2. Mengapa Origin C (127.0.0.1:8000) dianggap berbeda dari localhost:8000?	Browser membandingkan host secara literal. localhost dan 127.0.0.1 memang menuju mesin yang sama, tetapi tulisannya berbeda sehingga origin berbeda. Buktinya getItem('username') bernilai null pada Gambar 7.
Q3. Apa bukti bahwa UI browser bukan security boundary?	Input role yang disabled dapat diaktifkan dan diubah dari student menjadi admin lewat Console tanpa izin server (Gambar 8). Tombol Submit membaca nilai dari DOM, jadi logika client bisa dipengaruhi pengguna.
Q4. Minimal 5 bagian request/data yang dapat dikontrol client.	(1) URL dan path, (2) query string (mis. ?role=admin&debug=true), (3) header HTTP (mis. X-Student-Lab, User-Agent), (4) method (GET, HEAD, POST), (5) body/form data, (6) isi localStorage dan DOM.
Q5. Jika role disimpan di localStorage, kontrol apa yang tetap wajib di server?	Server tetap harus melakukan authentication dan authorization sendiri pada setiap endpoint, mengambil role dari sumber tepercaya (mis. session atau token yang diverifikasi server), dan memvalidasi ulang nilai sensitif. Role dari client dianggap bisa dipalsukan.
Q6. Beda validation untuk usability dan untuk security?	Validasi usability (field disabled, format input) memberi umpan balik cepat bagi pengguna tetapi bisa dilewati. Validasi security dilakukan di server, wajib untuk semua input, dan tidak bergantung pada UI.

7. Kesimpulan

1. HTTP client: browser dan curl sama-sama mengirim request; request bisa dibuat di luar UI resmi.
2. Origin: ditentukan oleh scheme + host + port; path dan query tidak termasuk.
3. Storage isolation: localStorage terpisah per origin, sehingga data Origin A tidak terbaca dari 127.0.0.1:8000.
4. DOM: elemen UI dan state client dapat diubah pengguna (role student menjadi admin).
5. Trust boundary: server harus menganggap input client tidak tepercaya dan menegakkan validation, authentication, dan authorization secara independen.